

ShazamKit Custom-Catalog

API Use Cases

Uses code samples from:

Matcher.swift

CatalogProvider.swift

Episode.swift

Question.swift



Found in
FoodMath ShazamKit sample
from Apple Developer

Scope:

This document distinguishes

- (1) behavior directly demonstrated by the uploaded FoodMath files
- (2) behavior confirmed in Apple ShazamKit documentation
- (3) illustrative application code adapted for the two proposed use cases.

Use Case **A**

Recognize external audio to open a door, unlock a treasure chest, or advance a level in a video game

Use Case **B**

Recognize an ad jingle or commercial track and navigate the user to a product destination

Contents

- 1. Overview of Concept 3
- 2. FoodMath (Apple Developer package) Abstraction..... 4
- 3. API Reference 5
- 4. Source-file responsibilities 5
- 5. Use Case A - Video game event triggers 6
- 6. Use Case B - Shoppable Audio Marketplace 8
- 7. Shared custom-catalog design 10
- 8. Error handling, privacy, and implementation boundaries 11
- 9. Testing strategy 12
- 10. Recommended project structure 13
- 11. Source notes 13

Make interactive table contents

1. Overview of Concept

The uploaded FoodMath sample implements an event-driven ShazamKit pattern that can be generalized well beyond educational content. The sample creates a custom catalog, listens to live audio, submits streaming audio buffers to a ShazamKit session, receives a match through SHSessionDelegate, and then updates application state. The same sequence can power both of the proposed concepts.

Stage	FoodMath sample	Video game adaptation	Marketplace adaptation
Reference audio	FoodMath.shazamsignature	Song / sound cue / licensed recording signature	Authorized jingle or commercial-music signature
Associated metadata	Title, subtitle, episode, teacher	gameEvent, objectID, levelID	brand, productID, productURL
Live input	AVAudioEngine microphone input	Environmental audio heard by game companion device	Commercial/jingle heard by phone
Match callback	session(_:didFind:)	Translate match to GameAction	Translate match to ProductDestination
Application effect	Update synchronized lesson/question	Open door, unlock chest, advance level	Show product and open approved retailer destination

Core principle

ShazamKit recognizes audio and returns metadata. It does not itself open a game object or complete a purchase. Your application converts the match result into a domain-specific action.

2. FoodMath (Apple Developer package)

Abstraction

The five source files divide into two groups: ShazamKit infrastructure and application-domain models. This distinction is central to adapting the sample safely and cleanly.

File	Role in the sample	What to reuse
Matcher.swift	Creates SHSession, captures audio with AVAudioEngine, streams buffers, receives SHMatch, publishes MatchResult.	Reuse or refactor as the recognition layer.
CatalogProvider.swift	Loads FoodMath.shazamsignature, creates SHSignature, associates SHMediaItem metadata, builds SHCustomCatalog.	Reuse as the custom-catalog construction pattern.
Question.swift	Models synchronized lesson questions and offsets.	Replace with GameEvent or ProductRecord models.
Episode.swift	Models lesson episodes.	Replace with game levels, campaigns, brands, or product groups as needed.
Equation.swift	Models the educational action/state used by the UI.	Replace with application-specific actions; it is not a ShazamKit requirement.

The code hides complicated implementation details behind simpler components.

Correlation

Question.swift

Defines / stores question data

Query friendly

Matcher.swift

Behaves like a lookup / query

searches that question based on the Shazam match

```
let newQuestion = Question.allQuestions.last { question in
    (match.mediaItems.first?.predictedCurrentMatchOffset ?? 0) >
    question.offset
}
```

2.1 Recognition infrastructure

`Matcher.swift` is explicitly described by the sample as the model responsible for matching against the catalog and updating SwiftUI views. It imports `ShazamKit`, `AVFAudio`, and `Combine`; creates an `SHSession` and `AVAudioEngine`; and passes microphone buffers to `matchStreamingBuffer(_:at:)`

```
session = SHSession(catalog: catalog)
session?.delegate = self

audioEngine.inputNode.installTap(
    onBus: 0,
    bufferSize: 2048,
    format: audioFormat
) { [weak session] buffer, audioTime in
    session?.matchStreamingBuffer(buffer, at: audioTime)
}
```

2.2 Catalog construction

`CatalogProvider.swift` loads a bundled `.shazamsignature` file, converts its data into `SHSignature`, creates an `SHMediaItem`, and adds that signature-plus-metadata pair to an `SHCustomCatalog`.

```
let signatureData = try Data(contentsOf: signaturePath)
let signature = try SHSignature(dataRepresentation: signatureData)

let mediaItem = SHMediaItem(properties: [
    .title: "Learn with Neil",
    .subtitle: "Count on Me",
    .episode: 3,
    .teacher: "Neil Foley"
])

let customCatalog = SHCustomCatalog()
try customCatalog.addReferenceSignature(
    signature,
    representing: [mediaItem]
)
```

Source-derived observation `FoodMath` proves the application pattern: a match can carry custom metadata that the rest of the app interprets. The proposed game and marketplace behaviors are adaptations of that pattern, not behaviors implemented in the uploaded sample itself.

`let mediaItem` contains more metadata about the song found in the Spotify catalog.

3. API Reference

Concept	Purpose	Why it matters here
SHSignature	Opaque acoustic signature representing reference audio.	Lets a custom catalog recognize your own authorized reference recordings.
SHCustomCatalog	Stores reference signatures and associated media-item metadata.	Creates a private recognition universe for game cues or campaign jingles.
SHMediaItem	Metadata attached to a reference signature.	Carries game-action identifiers or product destination data.
SHSession	Matches query audio against a catalog.	Connects captured audio to recognition results.
matchStreamingBuffer(_:at:)	Converts streaming audio buffers into query-signature data and searches the catalog.	Supports continuous environmental listening in the FoodMath-style architecture.
SHSessionDelegate	Receives asynchronous match results.	The natural place to dispatch game or marketplace events.
SHMatch / SHMatchedMediaItem	Represents matching media items and match-specific information.	Supplies the metadata your app interprets after recognition.
predictedCurrentMatchOffset	Estimated current position in the matched reference recording.	Can support time-segment-specific game actions or synchronized experiences.

Apple documentation note Apple documents that custom **SHMediaItem** properties can be added by extending **SHMediaItemProperty**. Supported custom-value object types include Dictionary, Array, URL, Number, String, Date, and Data.

ShazamKit uses a recognition pipeline that connects incoming audio to an application-specific result. A reference recording is first converted into an **SHSignature**, which acts as an acoustic fingerprint rather than storing the full audio itself. That signature is added to an **SHCustomCatalog** and associated with an **SHMediaItem**, which can contain metadata describing what the recognized audio means to the application. An **SHSession** then listens for incoming audio and processes it through **matchStreamingBuffer(_:at:)**, which continuously sends small audio buffers into the matching system. When ShazamKit recognizes a reference signature, it returns an **SHMatch** containing an **SHMatchedMediaItem**. At that point, the application can inspect the media item's metadata and decide what action to take.

This architecture is especially useful because ShazamKit handles the audio recognition, while the application controls the behavior that follows. In a video game, a matched media item could contain custom values such as `gameEvent = "openDoor"`, `objectID = "castleDoor01"`, or `levelID = "level3"`. When the session delegate receives the match, the game could read those values and call functions such as `openDoor()`, `unlockChest()`, or `advanceToNextLevel()`. In a marketplace application, the same pattern could associate a commercial jingle with metadata such as a brand name, product identifier, and product URL. After recognizing the jingle, the app could display the corresponding product and provide a link to an approved retailer or marketplace destination. In both cases, ShazamKit identifies the audio; the metadata explains what the match represents; and the application uses that information to trigger the appropriate event.

4. Source-file responsibilities

3.1 Matcher.swift - the recognition engine

1. Create an `SHSession` using the supplied custom catalog.
2. Set `Matcher` as the session delegate so it receives match callbacks.
3. Create a mono `AVAudioFormat` and install a tap on the audio-engine input node.
4. Send each captured buffer to `matchStreamingBuffer(_:at:)`.
5. Request microphone permission and start `AVAudioEngine`.
6. When `session(_:didFind:)` fires, inspect the matched media item and update application state on the main queue.

3.2 CatalogProvider.swift - the reference data layer

`CatalogProvider.swift` is where the sample establishes the relationship between an audio signature and human-readable/application-readable metadata. For the proposed systems, this file is the main place to change what a recognized sound means.

3.3 Question.swift, Episode.swift, Equation.swift - replaceable domain layer

These three files are specific to FoodMath's learning experience. `Question` adds time offsets and answer rules; `Episode` groups educational content; `Equation` models math operations. For a game, they can be replaced by `GameEvent`, `GameLevel`, and `GameAction` models. For commerce, they can be replaced by `ProductRecord`, `Campaign`, and `ProductDestination` models.

4. Use Case A - Video game event triggers

Goal: an external song or sound is played in the physical environment. A ShazamKit-enabled companion app or supported Apple-platform game captures the sound, matches it against a custom catalog, and converts the result into a game event such as opening a door, unlocking a chest, or advancing to the next level.

Recognition result	Example metadata	Game response
Door song matched	gameEvent = openDoor; objectID = door_castle_01	Set the matching door state to open.
Treasure cue matched	gameEvent = unlockChest; objectID = chest_vault_07	Unlock or animate the target chest.
Level-completion song matched	gameEvent = nextLevel; levelID = temple_03	Advance progression after validating game state.

Example:

ShazamGameTriggers.swift

```
import Foundation
import ShazamKit
import AVFoundation

extension SHMediaItemProperty {
    static let gameEvent = SHMediaItemProperty("gameEvent")
    static let objectID = SHMediaItemProperty("objectID")
    static let levelID = SHMediaItemProperty("levelID")
}

enum GameTriggerEvent {
    case openDoor(objectID: String)
    case unlockChest(objectID: String)
    case nextLevel(levelID: String)
    case unrecognized(raw: String)

    /// Fails to initialize if the matched media item is missing the fields
    /// required for its declared gameEvent.
    init?(mediaItem: SHMediaItem) {
        guard let event = mediaItem.gameEvent else { return nil }

        switch event {
        case "openDoor":
            guard let objectID = mediaItem.objectID else { return nil }
            self = .openDoor(objectID: objectID)

        case "unlockChest":
            guard let objectID = mediaItem.objectID else { return nil }
            self = .unlockChest(objectID: objectID)
```

```

        case "nextLevel":
            guard let levelID = mediaItem.levelID else { return nil }
            self = .nextLevel(levelID: levelID)

        default:
            self = .unrecognized(raw: event)
        }
    }
}

private let referenceSounds: [ReferenceSound]
private let cacheURL: URL

init(referenceSounds: [ReferenceSound],
      cacheDirectory: URL = FileManager.default.urls(for: .cachesDirectory, in:
.userDomainMask)[0]) {
    self.referenceSounds = referenceSounds
    self.cacheURL =
cacheDirectory.appendingPathComponent("GameTriggers.shazamcatalog")
}

```

```
// this is an important line
```

```

        .gameEvent: sound.gameEvent

    // Cache the built catalog so the app doesn't regenerate signatures
    // (a somewhat expensive, file-reading operation) on every launch.
    try? catalog.dataRepresentation.write(to: cacheURL, options: .atomic)

    return catalog
}

while true {
    buffer.frameLength = 0
    try file.read(into: buffer, frameCount: frameCount)
}

```

```

        if buffer.frameLength == 0 { break }
        try generator.append(buffer, at: nil)
    }

    return generator.signature()
}
}

var onNoMatch: (() -> Void)?

private var session: SHSession?
private let audioEngine = AVAudioEngine()
private(set) var isListening = false

/// Starts listening for the game's custom audio cues.
/// - Parameter catalog: A catalog built via `GameAudioCatalogBuilder`.

        .compactMap(GameTriggerEvent.init(mediaItem:))
        .first

guard let event else { return }

DispatchQueue.main.async { [weak self] in
    self?.onTrigger?(event)
}

}

func session(_ session: SHSession, didNotFindMatchFor signature: SHSignature, error:
Error?) {
    DispatchQueue.main.async { [weak self] in
        self?.onNoMatch?()
    }
}

}
}

```

```

protocol GameWorld: AnyObject {
    func setDoorState(objectID: String, open: Bool)
    func unlockChest(objectID: String)
    var currentLevelID: String { get }
    func isLevelComplete(_ levelID: String) -> Bool
    func advance(toLevel levelID: String)
}

final class GameEventDispatcher {
    private weak var world: GameWorld?

    init(world: GameWorld) {
        self.world = world
    }

    func handle(_ event: GameTriggerEvent) {
        guard let world else { return }

        switch event {
        case .openDoor(let objectID):
            world.setDoorState(objectID: objectID, open: true)

        case .unlockChest(let objectID):
            world.unlockChest(objectID: objectID)

        case .nextLevel(let levelID):
            guard world.currentLevelID == levelID, world.isLevelComplete(levelID) else {
                print("Ignoring nextLevel trigger for \(levelID): validation failed.")
                return
            }
            world.advance(toLevel: levelID)

        case .unrecognized(let raw):

```

```

        print("Matched a reference sound with an unhandled gameEvent: \(raw)")
    }
}

```

```

final class DemoGameWorld: GameWorld {
    private(set) var doorStates: [String: Bool] = [:]
    private(set) var unlockedChests: Set<String> = []
    private(set) var currentLevelID: String = "temple_03"
    private var completedLevels: Set<String> = ["temple_03"]

```

```

// pretend the player already cleared it

```

```

    func setDoorState(objectID: String, open: Bool) {
        doorStates[objectID] = open
        print("[World] Door \(objectID) is now \(open ? "OPEN" : "CLOSED").")
    }

```

```

    func unlockChest(objectID: String) {
        unlockedChests.insert(objectID)
        print("[World] Chest \(objectID) unlocked.")
    }

```

```

    func isLevelComplete(_ levelID: String) -> Bool {
        completedLevels.contains(levelID)
    }

```

```

    func advance(toLevel levelID: String) {
        currentLevelID = levelID
        print("[World] Advanced to next level.")
    }

```

```

}

```

```

/// Example orchestration – wire this into your app / scene's startup code.

```

```

@MainActor

```

```

final class GameAudioTriggerController {
    private let world = DemoGameWorld()
    private lazy var dispatcher = GameEventDispatcher(world: world)
    private let triggerManager = ShazamGameTriggerManager()

    func start() async {
        // 1. Ask for mic permission up front (Info.plist needs
        //     NSMicrophoneUsageDescription).
        let granted = await AVAudioApplication.requestRecordPermission()
        guard granted else {
            print("Microphone permission denied; cannot listen for game cues.")
            return
        }

        // 2. Describe the reference sounds that make up this level's audio
        //     puzzle. These would ship as bundled audio assets.
        let referenceSounds: [GameAudioCatalogBuilder.ReferenceSound] = [
            .init(resourceName: "door_castle_01_cue", resourceExtension: "caf",
                  gameEvent: "openDoor", objectID: "door_castle_01", levelID: nil),
            .init(resourceName: "chest_vault_07_cue", resourceExtension: "caf",
                  gameEvent: "unlockChest", objectID: "chest_vault_07", levelID: nil),
            .init(resourceName: "temple_03_complete_cue", resourceExtension: "caf",
                  gameEvent: "nextLevel", objectID: nil, levelID: "temple_03"),
        ]

        do {
            // 3. Build (or load the cached) custom catalog.
            let builder = GameAudioCatalogBuilder(referenceSounds: referenceSounds)
            let catalog = try await builder.buildOrLoadCatalog()

            // 4. Route matches to the dispatcher.
            triggerManager.onTrigger = { [weak self] event in
                self?.dispatcher.handle(event)
            }
            triggerManager.onNoMatch = {

```

```
        print("Heard something, but it didn't match a known game cue.")
    }

    // 5. Start listening.
    try triggerManager.startListening(with: catalog)
    print("Listening for in-world audio cues...")

} catch {
    print("Failed to start audio trigger system: \(error)")
}

func stop() {
    triggerManager.stopListening()
}
}
```

Shorten this code sample

5.1 Define custom media-item property keys

Apple's documented extension pattern is a clean way to create readable custom metadata keys. The following code is illustrative adaptation code for the game use case.

```
import ShazamKit

extension SHMediaItemProperty {
    static let gameEvent = SHMediaItemProperty("GameEvent")
    static let objectID = SHMediaItemProperty("ObjectID")
    static let levelID = SHMediaItemProperty("LevelID")
}

extension SHMediaItem {
    // question marks mean the properties are optionals
    var gameEvent: String? { self[.gameEvent] as? String }
    var objectID: String? { self[.objectID] as? String }
    var levelID: String? { self[.levelID] as? String }
}
```

5.2 Build the game custom catalog

```
struct GameCatalogProvider {
    static func catalog() throws -> SHCustomCatalog {
        let catalog = SHCustomCatalog()

        guard let doorURL = Bundle.main.url(
            forResource: "CastleDoorSong",
            withExtension: "shazamsignature"
        ) else {
            throw GameCatalogError.missingSignature
        }

        let data = try Data(contentsOf: doorURL)
        let signature = try SHSignature(dataRepresentation: data)

        let item = SHMediaItem(properties: [
            .title: "Castle Door Song",
            .gameEvent: "openDoor",
            .objectID: "door_castle_01",
            .levelID: "castle_01"
        ])

        try catalog.addReferenceSignature(
            signature,
            representing: [item]
        )

        return catalog
    }
}

enum GameCatalogError: Error {
    case missingSignature
}
```

Implementation note

The FoodMath file loads one FoodMath.shazamsignature. A real game catalog could add multiple signatures, each with different metadata, before the catalog is attached to an SHSession.

In completed API code for hypothetical video game, you would add more let item songs to the library before sorting them for activation by the video game mic.

5.3 Translate a match into a game action

```
enum GameAction {
    case openDoor(objectID: String)
    case unlockChest(objectID: String)
    case nextLevel(levelID: String)
}

func gameAction(from item: SHMatchedMediaItem) -> GameAction? {
    switch item.gameEvent {
    case "openDoor":
        guard let id = item.objectID else { return nil }
        return .openDoor(objectID: id)

    case "unlockChest":
        guard let id = item.objectID else { return nil }
        return .unlockChest(objectID: id)

    case "nextLevel":
        guard let id = item.levelID else { return nil }
        return .nextLevel(levelID: id)

    default:
        return nil
    }
}
```

5.4 Adapt Matcher.session(_:didFind:)

```
func session(_ session: SHSession, didFind match: SHMatch) {
    guard let item = match.mediaItems.first,
          let action = gameAction(from: item) else {
        return
    }

    DispatchQueue.main.async {
        self.dispatch(action)
    }
}

private func dispatch(_ action: GameAction) {
    switch action {
    case .openDoor(let objectID):
        gameState.openDoor(id: objectID)

    case .unlockChest(let objectID):
        gameState.unlockChest(id: objectID)

    case .nextLevel(let levelID):
        gameState.advance(to: levelID)
    }
}
```

Important boundary `gameState.openDoor(...)`, `unlockChest(...)`, and `advance(...)` are application methods you would write.

ShazamKit produces the recognition event; the game engine owns the world-state change.

5.5 Optional: use match offset for time-sensitive triggers

FoodMath uses `predictedCurrentMatchOffset` to choose the last Question whose offset is earlier than the current match position. The same strategy can create segment-specific game events inside one longer reference recording.

```
let currentOffset =  
    match.mediaItems.first?.predictedCurrentMatchOffset ?? 0  
  
if currentOffset >= 0 && currentOffset < 20 {  
    dispatch(.openDoor(objectID: "door_a"))  
} else if currentOffset >= 20 && currentOffset < 40 {  
    dispatch(.unlockChest(objectID: "chest_b"))  
}
```

Design caution Avoid firing the same event repeatedly while the same audio continues to match. FoodMath already demonstrates de-duplication by comparing the current Question with the newly derived Question before publishing a new result. A game should implement a similar event-state guard.

5.6 Remote-control interpretation

A television or game remote can initiate a “listen now” state, but the remote is not automatically the ShazamKit execution environment. In the FoodMath architecture, the device running `AVAudioEngine` captures environmental sound.

A practical design is:

remote/button command → companion iPhone/iPad begins listening → ShazamKit matches →
game receives or observes the resulting action

**ADD MORE ABOUT HOW THIS WORKS IN TERMS
OF WHERE THE SONG IS COMING FROM ETC**

5. Use Case B - Shoppable Audio Marketplace

Goal: an authorized advertisement jingle or commercial-music recording is represented in a custom Shazam catalog. When the user's phone recognizes that audio, the app displays product information and gives the user a deliberate action that opens an approved retailer or marketplace destination.

Example framing "Clorox ad jingle → recognize → show Clorox product → user taps View Product → approved retailer/app/web destination." This is an illustrative architecture. Actual brand audio, trademarks, product data, retailer links, affiliate links, and music require the appropriate permissions and commercial agreements.

5.1 Define product metadata properties

```
import ShazamKit

extension SHMediaItemProperty {
    static let brand = SHMediaItemProperty("Brand")
    static let productID = SHMediaItemProperty("ProductID")
    static let productName = SHMediaItemProperty("ProductName")
    static let productURL = SHMediaItemProperty("ProductURL")
    static let campaignID = SHMediaItemProperty("CampaignID")
}

extension SHMediaItem {
    var brand: String? { self[.brand] as? String }
    var productID: String? { self[.productID] as? String }
    var productName: String? { self[.productName] as? String }
    var productURL: URL? { self[.productURL] as? URL }
    var campaignID: String? { self[.campaignID] as? String }
}
```

5.2 Associate a jingle with a product destination

```
let item = SHMediaItem(properties: [
    .title: "Example Cleaning Brand Campaign",
    .brand: "Example Brand",
    .productID: "wipe-001",
    .productName: "Disinfecting Wipes",
    .productURL: URL(
        string: "https://retailer.example/products/wipe-001"
    )!,
    .campaignID: "spring-campaign-2026"
])

try catalog.addReferenceSignature(
    signature,
    representing: [item]
)
```

Why the URL belongs in metadata Apple documents URL as a supported custom SHMediaItem value type. Storing a destination URL with the reference metadata allows the recognition layer to return enough

information for the app to build a product result.

5.3 Build a product result from SHMatch

```
struct MatchedProduct {
    let brand: String
    let name: String
    let productID: String
    let destination: URL
}

func product(from item: SHMatchedMediaItem) -> MatchedProduct? {
    guard let brand = item.brand,
          let name = item.productName,
          let id = item.productID,
          let url = item.productURL else {
        return nil
    }

    return MatchedProduct(
        brand: brand,
        name: name,
        productID: id,
        destination: url
    )
}
```

5.4 Update the UI; let the user choose to navigate

```
func session(_ session: SHSession, didFind match: SHMatch) {
    guard let item = match.mediaItems.first,
          let product = product(from: item) else {
        return
    }

    DispatchQueue.main.async {
        self.matchedProduct = product
    }
}
```

```
Button("View Product") {
    openURL(matchedProduct.destination)
}
```

Recommended interaction Recognition should normally surface the product first and let the user tap before leaving the app. That keeps the recognition event separate from the commerce/navigation decision and avoids surprising automatic

redirects.

6. Marketplace link behavior

ShazamKit is the recognition layer, not the Amazon/retailer commerce API. Whether a destination opens a marketplace app or a website depends on the retailer’s supported deep links, universal links, affiliate rules, API terms, and the user’s installed apps. Treat the destination as a separate integration boundary.

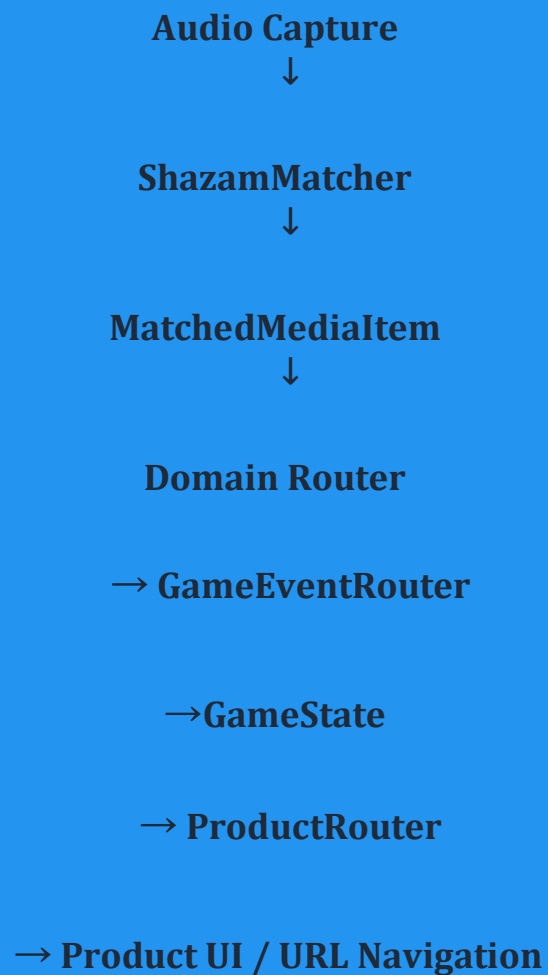
Layer	Responsibility
ShazamKit custom catalog	Recognize the authorized jingle/commercial recording.
SHMedialtem metadata	Associate the match with brand/product identifiers and an approved destination.
Your app	Display the matched product and capture the user’s navigation intent.
Retailer/marketplace	Resolve the URL/deep link, show the product, and handle cart/checkout under its own rules.

7. Shared custom-catalog design

Both proposed systems benefit from keeping recognition metadata declarative. Instead of hard-coding “this song opens this door” deep inside `Matcher.swift`, store identifiers in the catalog and let a separate router interpret them.

Metadata field	Game example	Marketplace example
title	Castle Door Song	Cleaning Campaign Jingle
type	gameEvent	product
action / destination	openDoor	https://retailer.example/...
target identifier	door_castle_01	wipe-001
group identifier	level_castle_01	spring-campaign-2026

7.1 Recommended separation of concerns



- **Matcher should know how to recognize audio, not how to animate a door or build a checkout experience.**
- **CatalogProvider should define the reference signatures and metadata association.**
- **Domain routers should validate metadata before dispatching an action.**
- **UI/game state should remain on the main actor/main queue when updates affect SwiftUI or gameplay state.**

8. Error handling, privacy, and implementation boundaries

Concern	Recommended handling
Microphone permission denied	Show a clear explanation and a retry/settings path; do not start recognition without permission.
Missing .shazamsignature	Fail catalog construction with a specific error rather than silently returning an empty experience.
No match	Keep listening or present “No match found”; do not dispatch an application action.
Malformed metadata	Reject the match for routing purposes if required IDs/URLs are missing or invalid.
Duplicate match callbacks	Track the last dispatched action or resolved media item and suppress repeated state changes.
Untrusted destination URL	Allowlist approved hosts/schemes before presenting or opening a commerce destination.
Audio/music rights	Use only reference audio and commercial/music assets for which the project has the necessary rights and permissions.
Automatic purchase/checkout	Do not treat an audio match as authorization to buy. Keep recognition, navigation, cart, and purchase consent distinct.

8.1 What FoodMath does not provide

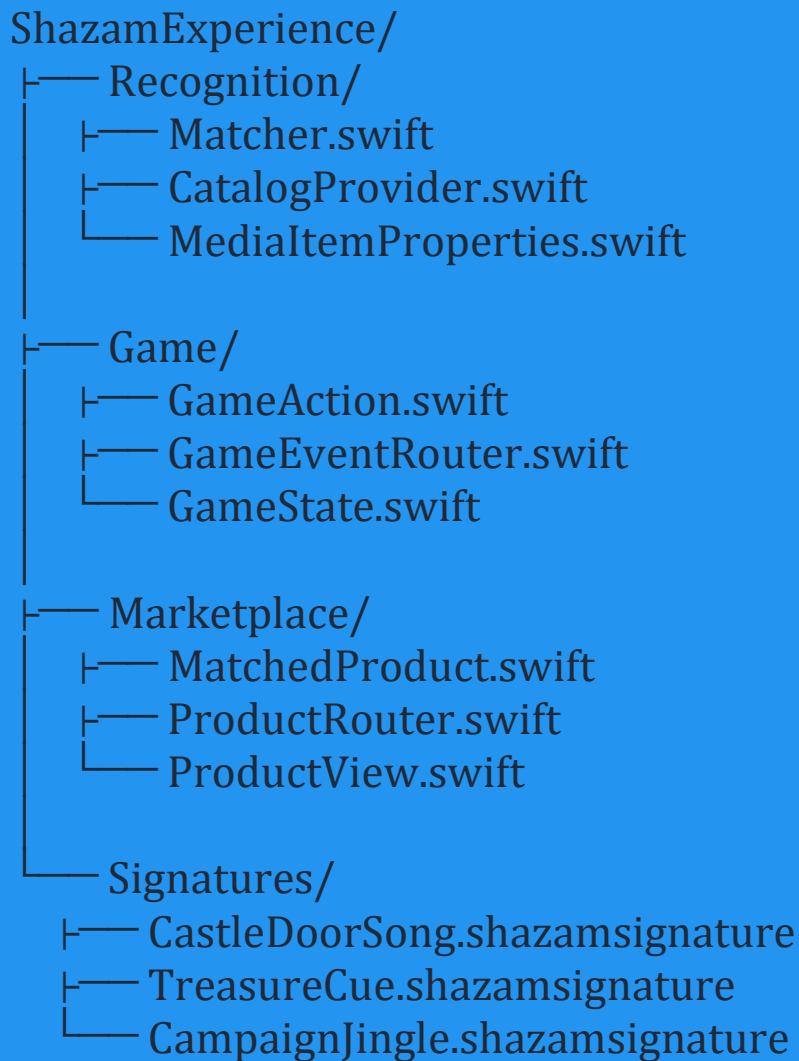
- A network protocol for sending a recognition result from an iPhone to a separate console/game process.
- A game-engine integration such as SceneKit, SpriteKit, Unity, Unreal, or a proprietary console engine.
- Retailer-specific product APIs, affiliate APIs, checkout flows, or Amazon-specific deep-link rules.
- Rights management for commercial jingles, songs, trademarks, or brand assets.
- Server-side catalog management, analytics, campaign attribution, inventory lookup, or product-price synchronization.

Why this matters The uploaded files are a strong recognition-and-state-update sample. Networking, game-engine state, and commerce are downstream systems that must be designed separately.

9. Testing strategy

Test	Expected result
Play the exact reference audio in a quiet room	Correct media item appears; correct action/product is derived.
Play a different catalog item	No incorrect action from the prior item; correct new metadata is used.
Play unrelated audio	No game action or product navigation is produced.
Repeat the same song for 30 seconds	Duplicate-action guard prevents the door/chest/level event from firing continuously.
Start playback in the middle of a long reference	Match still resolves when enough audio is captured; offset-based logic uses the returned match position where appropriate.
Deny microphone permission	App handles denial without crashing or silently pretending to listen.
Product URL points to an unapproved host	Navigation layer rejects or disables the destination.
Required custom property missing	Router rejects the incomplete result safely.

10. Recommended project structure



Portfolio recommendation

For a technical-writing portfolio, present `Matcher.swift` and `CatalogProvider.swift` as the reusable recognition infrastructure, then show the two domain routers as separate examples of how the same API result can drive very different user experiences.

11. Source notes

11.1 Uploaded FoodMath files

File	Size	Relevant content
Matcher.swift	63 lines	Matching model; SHSessionDelegate; AVAudioEngine; matchStreamingBuffer; predictedCurrentMatchOffset; MatchResult update.
CatalogProvider.swift	28 lines	Loads FoodMath.shazamsignature; constructs SHSignature and SHMediaItem; adds signature to SHCustomCatalog.
Episode.swift	26 lines	Episode model and sample episode list.
Question.swift	33 lines	Question model, TimeInterval offset, answer rules, comparison/equality.
Equation.swift	86 lines	EquationType and EquationOperation domain logic for the educational experience.

11.2 Apple Developer Documentation used for verification

- **ShazamKit framework overview:** <https://developer.apple.com/documentation/shazamkit>
- **SHSession:** <https://developer.apple.com/documentation/shazamkit/shsession>
- **SHCustomCatalog:** <https://developer.apple.com/documentation/shazamkit/shcustomcatalog>
- **SHMediaItem:** <https://developer.apple.com/documentation/shazamkit/shmediaitem>
- **Building a Custom Catalog and Matching Audio:** <https://developer.apple.com/documentation/shazamkit/building-a-custom-catalog-and-matching-audio>

Document status: FoodMath-derived architecture + Apple-documented API behaviors + clearly labeled illustrative adaptations for the proposed video-game and marketplace concepts.